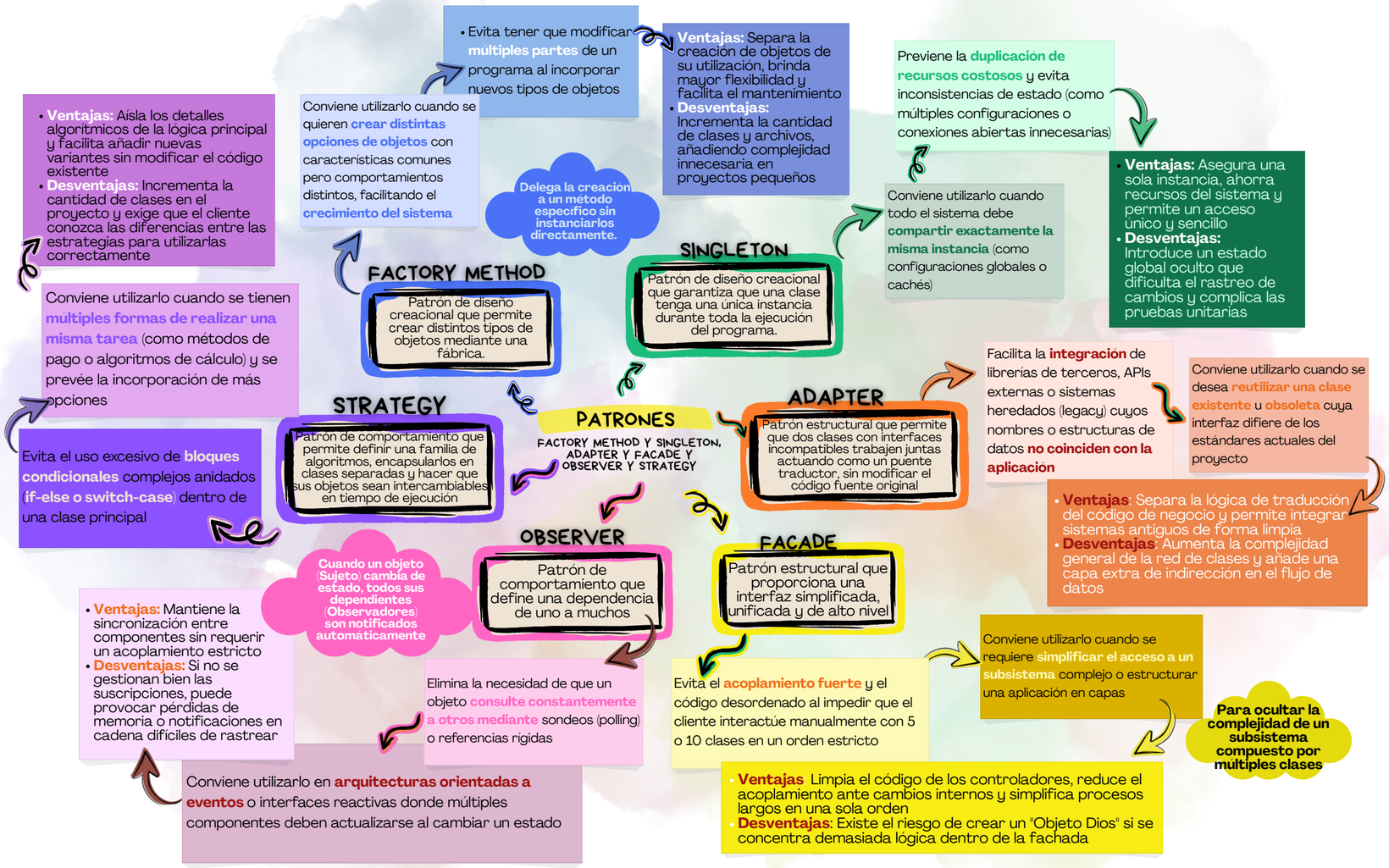

PATRONES
FACTORY METHOD Y SINGLETON, ADAPTER Y FACADE Y OBSERVER Y STRATEGY
FACTORY METHOD
Patrón de diseño creacional que permite crear distintos tipos de objetos mediante una fábrica.
Delega la creacion a un metodo especifico sin instanciarlos directamente.
Conviene utilizarlo cuando se quieren crear distintas opciones de objetos con características comunes pero comportamientos distintos, facilitando el crecimiento del sistema
• Evita tener que modificar múltiples partes de un programa al incorporar nuevos tipos de objetos
Ventajas: Separa la creación de objetos de su utilización, brinda mayor flexibilidad y facilita el mantenimiento
• Desventajas: Incrementa la cantidad de clases y archivos, añadiendo complejidad innecesaria en proyectos pequeños
• Ventajas: Aísla los detalles algorítmicos de la lógica principal y facilita añadir nuevas variantes sin modificar el código existente
• Desventajas: Incrementa la cantidad de clases en el proyecto y exige que el cliente conozca las diferencias entre las estrategias para utilizarlas correctamente
Conviene utilizarlo cuando se tienen multiples formas de realizar una misma tarea (como métodos de pago o algoritmos de cálculo) y se prevée la incorporación de más opciones
SINGLETON
Patrón de diseño creacional que garantiza que una clase tenga una única instancia durante toda la ejecución del programa.
Previene la duplicación de recursos costosos y evita inconsistencias de estado (como múltiples configuraciones o conexiones abiertas innecesarias)
Conviene utilizarlo cuando todo el sistema debe compartir exactamente la misma instancia (como configuraciones globales o cachés)
• Ventajas: Asegura una sola instancia, ahorra recursos del sistema y permite un acceso único y sencillo
• Desventajas: Introduce un estado global oculto que dificulta el rastreo de cambios y complica las pruebas unitarias
STRATEGY
Patrón de comportamiento que permite definir una familia de algoritmos, encapsularlos en clases separadas y hacer que sus objetos sean intercambiables en tiempo de ejecución
Evita el uso excesivo de bloques condicionales complejos anidados (if-else o switch-case) dentro de una clase principal
ADAPTER
Patrón estructural que permite que dos clases con interfaces incompatibles trabajen juntas actuando como un puente traductor, sin modificar el código fuente original
Facilita la integración de librerías de terceros, APIs externas o sistemas heredados (legacy) cuyos nombres o estructuras de datos no coinciden con la aplicación
Conviene utilizarlo cuando se desea reutilizar una clase existente u obsoleta cuya interfaz difiere de los estándares actuales del proyecto
• Ventajas: Separa la lógica de traducción del código de negocio y permite integrar sistemas antiguos de forma limpia
• Desventajas: Aumenta la complejidad general de la red de clases y añade una capa extra de indirección en el flujo de datos
OBSERVER
Patrón de comportamiento que define una dependencia de uno a muchos
Cuando un objeto (Sujeto) cambia de estado, todos sus dependientes (Observadores) son notificados automaticamente
• Ventajas: Mantiene la sincronización entre componentes sin requerir un acoplamiento estricto
• Desventajas: Si no se gestionan bien las suscripciones, puede provocar pérdidas de memoria o notificaciones en cadena difíciles de rastrear
Elimina la necesidad de que un objeto consulte constantemente a otros mediante sondeos (polling) o referencias rígidas
Conviene utilizarlo en arquitecturas orientadas a eventos o interfaces reactivas donde múltiples componentes deben actualizarse al cambiar un estado
FACADE
Patrón estructural que proporciona una interfaz simplificada, unificada y de alto nivel
Evita el acoplamiento fuerte y el código desordenado al impedir que el cliente interactúe manualmente con 5 o 10 clases en un orden estricto
Conviene utilizarlo cuando se requiere simplificar el acceso a un subsistema complejo o estructurar una aplicación en capas
Para ocultar la complejidad de un subsistema compuesto por múltiples clases
• Ventajas: Limpia el código de los controladores, reduce el acoplamiento ante cambios internos y simplifica procesos largos en una sola orden
• Desventajas: Existe el riesgo de crear un "Objeto Dios" si se concentra demasiada lógica dentro de la fachada